MCMP117 · Week 6 · Course Notes

Fundamentals of Operating Systems

Memory Management: Contiguous Allocation, Paging & Segmentation

Many programs, one RAM: how does every process get its own safe piece of memory?

Instructor: Asst. Prof. Dr. (Ph.D.) Hasan Oğuz
Head of the Electronical Communication Technology Programme, Vocational School
Istanbul Okan University

What you should be able to do after this week

1. Explain why an OS must manage memory, and distinguish compile-time, load-time and execution-time address binding.
2. Distinguish logical (virtual) from physical addresses and describe how an MMU with base (relocation) and limit registers translates and protects addresses, in decimal and in hexadecimal.
3. Compute swap times and explain why modern systems swap pages rather than whole processes.
4. Apply first-fit, best-fit and worst-fit to a hole list; explain external and internal fragmentation, compaction and the 50-percent rule.
5. Split an address into page number and offset, translate it through a page table, compute page-table sizes and internal fragmentation.
6. Compute the effective access time of a paged memory with a TLB.
7. Translate a segmented address $\langle s, d \rangle$ with a segment table, compare paging with segmentation, and say what x86-64 Linux actually uses.
8. Run small C programs that show virtual addresses, the page size and the memory map of a Linux process.

Contents

1 Why Memory Management? 1

2 Addresses and the Memory-Management Unit 2

3 Swapping 4

4 Contiguous Memory Allocation 5

5 Paging 8

6 Segmentation 13

7 Hands-on: Real Addresses on Linux 14

8 Summary 18

9 Glossary 18

10 Try It Yourself 20

11 Review Questions 20

12 Looking Ahead: Week 7 21

1 Why Memory Management?

Last week (Week 5) we studied how the operating system shares the *CPU* between processes: scheduling algorithms such as FCFS, SJF/SRTF, Priority and Round Robin decide who runs next, and mutexes and semaphores prevent race conditions when threads share data. In the language of Week 2 that was *time multiplexing*. This week is about *space multiplexing*: sharing **main memory** (RAM) between many processes at the same time.

Recall two facts from earlier weeks:

- **The storage hierarchy (Week 2).** The CPU can execute instructions and load data only from registers and main memory (caches hold copies of RAM). Disks and SSDs are reached only through I/O. A program must therefore be (at least partly) in RAM to run.
- **The process memory layout (Week 3).** Every process sees its memory as *text* (the code), *data* (global variables), the *heap* (`malloc`) and the *stack* (local variables, return addresses).

With multiprogramming, many such processes live in RAM simultaneously. The operating system, helped by the hardware, must solve three problems:

1. **Allocation.** Which part of RAM does each process get? What happens when RAM is full?
2. **Protection.** A process must not read or overwrite the memory of another process or of the kernel. Since a program touches memory hundreds of millions of times per second, every access must be checked, and that is only possible in *hardware*.
3. **Relocation.** A program does not know in advance where it will be loaded, and the OS may want to move it later.

2 Addresses and the Memory-Management Unit

2.1 Address binding

In a C program you write `count = count + 1;`. The variable `count` is a *name*; the CPU needs a *number*, the address of the bytes that hold `count`. Turning names into addresses is called **address binding**. On the way from source code to a running process, a program passes through several tools, and binding can be done at three different times.

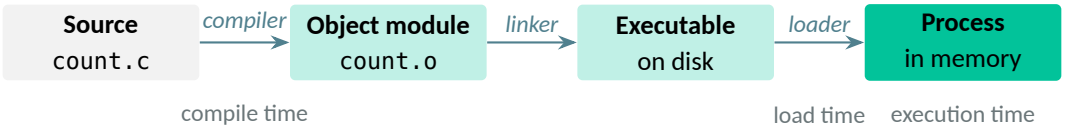


Figure 1: From source code to a running process.

Compile time
If it is known at compile time where the process will reside in memory, the compiler can generate **absolute code**. If the starting location changes, the program must be recompiled. MS-DOS `.COM` programs worked this way.

Load time
Otherwise, the compiler generates **relocatable code**: addresses such as "14 bytes from the beginning of this module". The loader converts them into absolute addresses when the program is brought into memory. After loading, the program cannot move.

Execution time
If the process may be moved during its execution, binding must be delayed until the instruction actually runs: every address is translated by hardware on every memory access. This needs special hardware, the MMU, but it is what all modern general-purpose operating systems do.

2.2 Logical versus physical addresses

An address generated by the CPU while executing a program is a **logical address**; with execution-time binding it is also called a **virtual address**. The address actually seen by the memory unit, placed on the memory bus, is a **physical address**. The set of all logical addresses a program may generate is its **logical address space**; the corresponding set of physical addresses is its **physical address space**.

With compile-time and load-time binding, logical and physical addresses are identical. With execution-time binding they differ, and the run-time translation is performed by a hardware device called the **memory-management unit** (MMU). The user program never sees physical addresses: every pointer you print with `printf("%p")` is logical. We will prove this experimentally in Section 7.

2.3 The simplest MMU: a relocation register

The simplest MMU contains one **relocation register**, also called the **base register**. Its value is added to every address generated by a user process at the moment it is sent to memory.

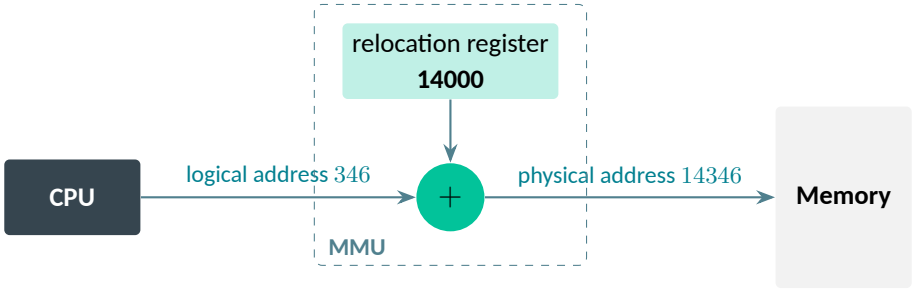


Figure 2: Dynamic relocation using a relocation register.

The program believes that it occupies addresses 0 to 2999. The OS has loaded it at physical address 14000 and written 14000 into the relocation register. When the CPU executes "load the word at address 346", the MMU sends $14000 + 346 = 14346$ to memory. To move the process, the OS copies its bytes elsewhere and changes one register. Loading the relocation register is a *privileged* instruction (Week 2): only the kernel may change it.

2.4 Protection with base and limit registers

Relocation alone does not protect anything: a program could use a very large logical address and reach another process. A second register, the **limit register**, holds the size of the logical address space. The MMU first checks the logical address against the limit, and only then adds the base:

$$\text{if } 0 \le a_{\text{log}} < \text{limit}: \quad a_{\text{phys}} = \text{base} + a_{\text{log}}; \qquad \text{otherwise: trap to the OS (addressing error).}$$

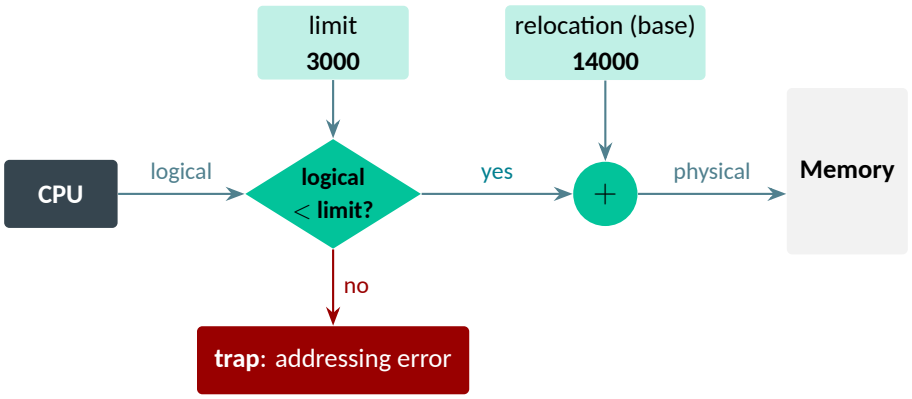


Figure 3: Hardware address protection with base and limit registers.

Because the check happens in hardware, on every access, in parallel with the addition, protection costs essentially no time. When the check fails, the CPU raises a trap (a software-generated interrupt, Week 2); the OS normally terminates the process. On Linux you see this as the message *Segmentation fault (core dumped)*; today it is produced by the paging hardware (Section 5.6), but the idea is the same.

Base and limit in decimal and in hexadecimal

Decimal. base $= 14000$, limit $= 3000$.

- $a = 346$: $346 < 3000$, so $a_{\text{phys}} = 14000 + 346 = 14346$.
- $a = 2999$: $2999 < 3000$, so $a_{\text{phys}} = 16999$, the last byte the process may use.
- $a = 3500$: $3500 \geq 3000$: **trap**.

The process owns exactly the physical range $[\text{base},\ \text{base} + \text{limit}) = [14000,\ 17000)$.
Hexadecimal. base $=$ `0x4000` $(= 16384)$, limit $=$ `0x1000` $(= 4096)$.

- $a =$ `0x02A4` $(= 676)$: `0x02A4` $<$ `0x1000`, so $a_{\text{phys}} =$ `0x4000` $+$ `0x02A4` $=$ `0x42A4`. Check in decimal: $16384 + 676 = 17060 =$ `0x42A4`.
- $a =$ `0x1200`: `0x1200` $\geq$ `0x1000`: **trap**.

Because the low three hex digits of the base are zero, the addition simply writes the offset into those digits. The legal physical range is `0x4000–0x4FFF`.

Base and limit in real machines

The CDC 6600 supercomputer (1964) protected each job with two registers called RA (reference address) and FL (field length), exactly the base/limit pair described here. The IBM System/360 used a different scheme (4-bit storage keys on 2 KB blocks). On today's PCs, the base/limit idea survives inside the segment descriptors of x86 processors (Section 6.4).

3 Swapping

If there is not enough RAM for all processes, the OS can temporarily move a process out of memory to a **backing store** (a fast disk or SSD) and bring it back later. This is **swapping**. The swapped-out process cannot run until it is swapped in again; if execution-time binding is used, it may come back to a different place in RAM.

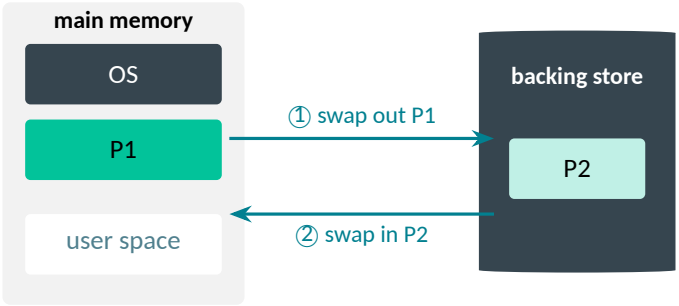


Figure 4: Swapping two processes using a disk as a backing store.

The cost of swapping is dominated by the *transfer time*, which is proportional to the amount of memory moved.

How long does a swap take?

A process occupies 100 MB; the backing store transfers 50 MB/s. Swapping it out takes

$$t_{\text{out}} = \frac{100\ \text{MB}}{50\ \text{MB/s}} = 2\ \text{s},$$

and swapping another process of the same size in takes another 2 s: about 4 s in total, ignoring disk latency. Compare with a Round-Robin time quantum of 10 ms (Week 5): one swap costs as much as $4\ \text{s}/10\ \text{ms} = 400$ quanta.
By contrast, moving a single 4 KB page at the same rate takes $4096\ \text{B}/(50 \times 10^6\ \text{B/s}) \approx 82\ \mu\text{s}$.

For this reason, standard whole-process swapping is no longer used by general-purpose systems except, at most, in emergencies. Linux, Windows and macOS instead swap individual *pages* (Section 5); the details, demand paging and page replacement, are the subject of Week 7. Mobile systems (iOS, Android) usually do not swap at all: when memory runs low they ask background apps to save their state and then terminate them.

4 Contiguous Memory Allocation

In **contiguous allocation**, each process occupies a single contiguous block of physical memory. With base and limit registers this is the natural scheme: one pair of numbers describes the whole process.

4.1 Variable partitions and the hole list

In the **variable-partition** scheme the OS gives each process a block of exactly the size it needs. Initially all user memory is one large free block. As processes arrive and terminate, memory becomes a patchwork of allocated blocks and free blocks called **holes**. The OS keeps a list of the holes. When a process arrives, the OS searches for a hole that is large enough, allocates what is needed and leaves the rest as a smaller hole. When a process terminates, its block becomes a hole, which is merged with any neighbouring holes.

Throughout this section we use one example. Memory has 2800 KB (a toy size, so that the numbers stay readable); the OS occupies the first 300 KB, five processes P1–P5 are resident, and there are five holes:

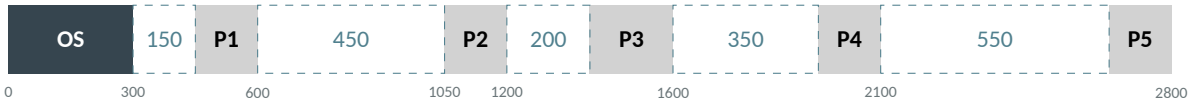


Figure 5: The running example: hole sizes in KB, addresses in KB below the bar.

The hole list, as (start, size) pairs in address order, is

$$(300, 150),\ (600, 450),\ (1200, 200),\ (1600, 350),\ (2100, 550), \qquad \text{total free} = 1700\ \text{KB}.$$

4.2 Placement algorithms: first-fit, best-fit, worst-fit

Which hole should a new process get? The three classical strategies are:

First-fit
Allocate the *first* hole that is big enough. Searching can stop as soon as one is found (from the start of the

list, or from where the last search ended, a variant called next-fit).

Best-fit
Allocate the *smallest* hole that is big enough. The whole list must be searched, unless it is kept sorted by size. Produces the smallest leftover hole.

Worst-fit
Allocate the *largest* hole. Also needs a full search. The idea is that the leftover remains large enough to be useful.

Four requests, three strategies

Processes arrive in the order A $=$ 220 KB, B $=$ 400 KB, C $=$ 130 KB, D $=$ 430 KB. Each request is carved from the beginning of the chosen hole.
Request A (220 KB). Holes large enough: 450, 350, 550. First-fit takes the first in address order, $450@600$; best-fit the smallest, $350@1600$; worst-fit the largest, $550@2100$.
Continuing in the same way:

Request	First-fit	Best-fit	Worst-fit
A = 220	$450@600 \rightarrow$ leaves 230	$350@1600 \rightarrow$ leaves 130	$550@2100 \rightarrow$ leaves 330
B = 400	$550@2100 \rightarrow$ leaves 150	$450@600 \rightarrow$ leaves 50	$450@600 \rightarrow$ leaves 50
C = 130	$150@300 \rightarrow$ leaves 20	$130@1820 \rightarrow$ leaves 0	$350@1600 \rightarrow$ leaves 220
D = 430	must wait	$550@2100 \rightarrow$ leaves 120	must wait

With first-fit, 950 KB are free when D arrives, but the largest hole is only 350 KB; with worst-fit the largest is 330 KB. Only best-fit places all four processes in this example.

The table was verified with a small Python script, `code/fit.py`. Its heart is the function that chooses a hole; the rest of the script just loops over the requests and prints the result:

```python
def choose(holes, size, rule):
    """Index of the hole chosen by the rule, or None if none is big enough."""
    fits = [i for i, (_, h) in enumerate(holes) if h >= size]
    if not fits:
        return None
    if rule == "first":
        return fits[0]                                  # lowest address
    if rule == "best":
        return min(fits, key=lambda i: holes[i][1])     # smallest that fits
    return max(fits, key=lambda i: holes[i][1])         # worst: largest
```

```
$ python3 fit.py

holes (size@start): 150@300, 450@600, 200@1200, 350@1600, 550@2100 | total = 1700 KB
requests: A=220, B=400, C=130, D=430

--- first-fit ---
  A (220 KB) -> hole 450@600, placed at 600-820, leftover 230 KB
  B (400 KB) -> hole 550@2100, placed at 2100-2500, leftover 150 KB
  C (130 KB) -> hole 150@300, placed at 300-430, leftover 20 KB
```

```
  D (430 KB): no hole large enough -> waits (free 950 KB, largest hole 350 KB)
  holes left: 20@430, 230@820, 200@1200, 350@1600, 150@2500 | total free = 950 KB
--- best-fit ---
  A (220 KB) -> hole 350@1600, placed at 1600-1820, leftover 130 KB
  B (400 KB) -> hole 450@600, placed at 600-1000, leftover 50 KB
  C (130 KB) -> hole 130@1820, placed at 1820-1950, leftover 0 KB
  D (430 KB) -> hole 550@2100, placed at 2100-2530, leftover 120 KB
  holes left: 150@300, 50@1000, 200@1200, 120@2530 | total free = 520 KB
--- worst-fit ---
  A (220 KB) -> hole 550@2100, placed at 2100-2320, leftover 330 KB
  B (400 KB) -> hole 450@600, placed at 600-1000, leftover 50 KB
  C (130 KB) -> hole 350@1600, placed at 1600-1730, leftover 220 KB
  D (430 KB): no hole large enough -> waits (free 950 KB, largest hole 330 KB)
  holes left: 150@300, 50@1000, 200@1200, 220@1730, 330@2320 | total free = 950 KB
```

One example proves nothing in general: best-fit tends to leave many tiny, useless holes (here 50 KB and, in other sequences, holes of a few bytes). Simulations reported by Silberschatz show that first-fit and best-fit are better than worst-fit in both time and storage utilisation; neither first-fit nor best-fit is clearly better in storage use, but first-fit is generally faster.

4.3 Fragmentation

External fragmentation. Enough total memory exists to satisfy a request, but it is not contiguous: it is split into many small holes. In the first-fit run, 950 KB were free but D (430 KB) could not be loaded. External fragmentation is typical of every scheme that hands out variable-size blocks (variable partitions, segmentation).

The 50-percent rule

Statistical analysis of first-fit (quoted by Silberschatz, after Knuth) shows that, even with some optimisation, if N blocks are allocated, another $0.5N$ blocks are lost to fragmentation. The fraction of memory that may be unusable is then

$$\frac{0.5N}{N + 0.5N} = \frac{1}{3}.$$

Internal fragmentation. Memory allocated to a process but not used by it: waste *inside* a block. It arises whenever memory is handed out in fixed-size units. Suppose a hole of $18\,464$ bytes is available and a process requests $18\,462$ bytes. Keeping a 2-byte hole in the list would cost more (list entry) than the hole itself, so the allocator gives away all $18\,464$ bytes; 2 bytes are internally fragmented. Internal fragmentation is typical of fixed partitions and of paging.

4.4 Compaction

One cure for external fragmentation is **compaction**: shuffle the memory contents so that all free memory is placed together in one large block. In our first-fit result the blocks OS, C, P1, A, P2, P3, P4, B, P5 occupy 1850 KB; moved to the low end of memory they leave a single hole of $2800 - 1850 = 950$ KB, and D fits (leaving 520 KB).

Compaction is possible only if relocation is dynamic (execution-time binding): the OS moves the bytes and updates the base register. It is also expensive: in the worst case all of memory is copied, the affected processes must be stopped, and a process doing I/O into a buffer cannot be moved while the device writes into that buffer. A better cure is to allow a process's memory to be *non-contiguous*. That is the idea of paging.

5 Paging

5.1 Pages and frames

Paging avoids external fragmentation completely. Physical memory is divided into fixed-size blocks called **frames**; logical memory is divided into blocks of the same size called **pages**. When a process is loaded, its pages are placed into *any* available frames. A per-process **page table** records, for each page, the frame that holds it. The OS keeps a list of free frames.

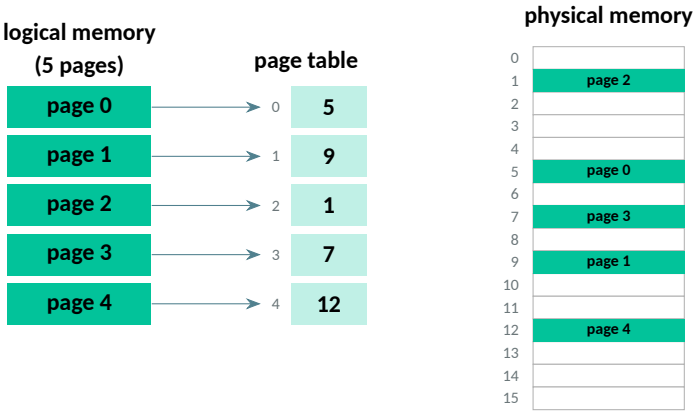


Figure 6: Paging: five pages of a process scattered over sixteen frames. This page table is used in all examples below and in `page_split.c`.

The page size is defined by the hardware and is always a power of 2, typically 4 KB (Linux on x86-64; `getconf PAGESIZE` prints 4096), sometimes 16 KB (Apple silicon) or larger.

5.2 Splitting an address into page number and offset

If the logical address space has 2^m bytes (m-bit addresses) and the page size is 2^n bytes, then the high-order $m - n$ bits of a logical address give the **page number** p, and the low-order n bits give the **page offset** d:



$$p = \left\lfloor \frac{a}{2^n} \right\rfloor = a \gg n, \qquad d = a \bmod 2^n = a \,\&\, (2^n - 1), \qquad a_{\text{phys}} = f \cdot 2^n + d,$$

where f is the frame number found in entry p of the page table. Because the page size is a power of 2, no division is needed: the hardware simply uses the upper bits as an index into the page table and copies the lower bits unchanged. For example, with 16-bit addresses and 4 KB pages ($n = 12$), p has 4 bits and the address space has $2^4 = 16$ pages; with 32-bit addresses, p has 20 bits (2^{20} pages).

A useful shortcut: one hexadecimal digit is exactly 4 bits, so with 4 KB pages the offset is the last three hex digits of the address.

Translating logical address `0x3A7F`

16-bit addresses, 4 KB pages, the page table of the figure above (page $0 \to 5$, $1 \to 9$, $2 \to 1$, $3 \to 7$, $4 \to 12$).
In binary, `0x3A7F` $= \underbrace{0011}_{p}\ \underbrace{1010\ 0111\ 1111}_{d}$, so $p = 3$ and $d =$ `0xA7F`.
In decimal: `0x3A7F` $= 14975$; $p = \lfloor 14975/4096 \rfloor = 3$, $d = 14975 - 3 \cdot 4096 = 2687 =$ `0xA7F`.
The page table gives frame $f = 7$, hence

$$a_{\text{phys}} = 7 \times 4096 + 2687 = 28672 + 2687 = 31359 = \texttt{0x7A7F}.$$

In hex, just replace the page digit 3 by the frame digit 7. Similarly, `0x1234` has $p = 1$, $d =$ `0x234`, frame 9, physical `0x9234` $= 37428$; and `0x5000` has $p = 5$, which is not part of the process: trap.

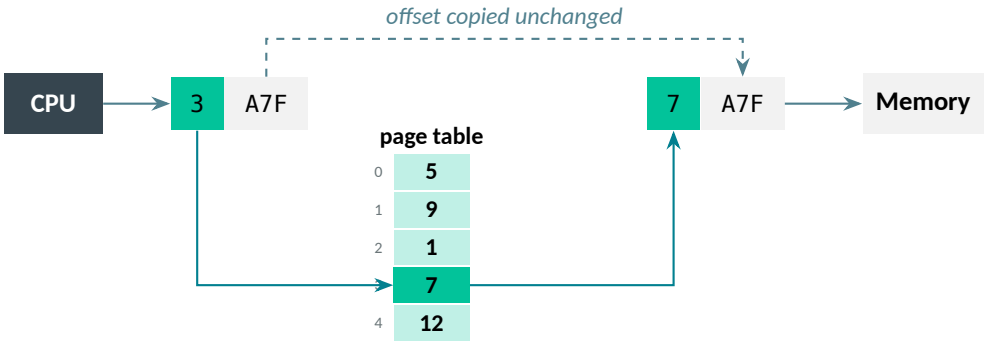


Figure 7: Paging hardware translating `0x3A7F` to `0x7A7F`.

5.3 Internal fragmentation in paging

Paging has no external fragmentation: any free frame can hold any page. But memory is handed out in whole frames, so the last frame of a process is usually not full.

A 10,000-byte process

With 4096-byte pages: $10\,000 = 2 \times 4096 + 1808$. The process needs 3 frames; the third holds only 1808 bytes, so $4096 - 1808 = 2288$ bytes are lost to internal fragmentation.

In the worst case a process needs n pages plus one byte, wasting almost a whole frame. If process sizes are independent of the page size, the last page is on average half full: the expected internal fragmentation is **half a page per process**. This argues for small pages; but smaller pages mean more pages, larger page tables and less efficient disk I/O. Page sizes have grown over time. Linux on x86-64 uses 4 KB pages normally, plus *huge pages* of 2 MB or 1 GB for programs with very large memory needs (our test machine reports `Hugepagesize: 2048 kB` in `/proc/meminfo`).

5.4 How large is a page table?

A page table has one entry per page of the logical address space.

Page-table size for a 32-bit address space

32-bit logical addresses, 4 KB pages, 4-byte page-table entries:

$$\text{entries} = \frac{2^{32}}{2^{12}} = 2^{20} = 1\,048\,576, \qquad \text{size} = 2^{20} \times 4\ \text{B} = 2^{22}\ \text{B} = 4\ \text{MB}$$

per process. With 100 processes that is 400 MB of page tables, most of which would describe unused parts of the address space. For 64-bit systems the problem is far worse: a flat table for a 48-bit address space with 8-byte entries would need $2^{36} \times 8$ B $= 512$ GB.

The solution, which we only outline, is the **multi-level** (hierarchical) page table: the page table is itself paged. In a two-level scheme for 32-bit addresses, the 20-bit page number is split into p_1 (10 bits) indexing an outer table of 2^{10} entries (exactly one 4 KB page) and p_2 (10 bits) indexing an inner table; inner tables exist only for the parts of the address space that are actually used. x86-64 uses four levels ($9+9+9+9$ bits of page number plus a 12-bit offset $= 48$ bits); recent processors support a fifth level (57-bit addresses). Each table at every level fits in exactly one 4 KB page (2^9 entries of 8 bytes).

5.5 The translation look-aside buffer (TLB)

The page table is kept in main memory, and a register (on x86, CR3) points to it. Then every data access needs *two* memory accesses: one for the page-table entry, one for the data itself. Paging would halve the speed of memory. The standard solution is a small, fast hardware cache inside the MMU, the **translation look-aside buffer** (TLB). Each TLB entry holds a (page number, frame number) pair; all entries are searched in parallel (associative memory). TLBs typically have between 32 and about 1536 entries.

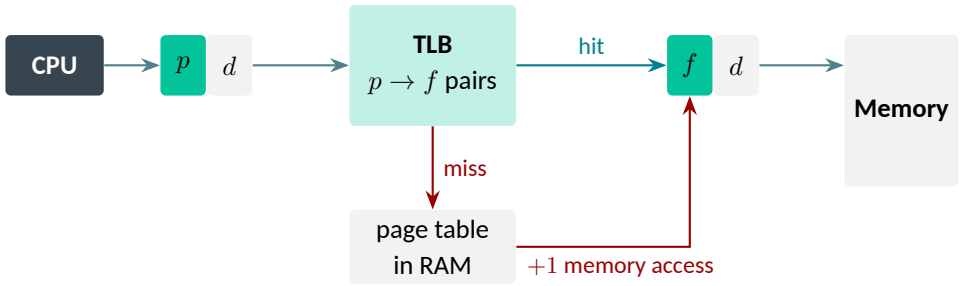


Figure 8: Paging hardware with a TLB.

On a *TLB hit* the frame number is available almost immediately. On a *TLB miss* the page-table entry must be read from memory (by hardware on x86, by the OS on some other architectures), and it is then stored in the TLB, possibly replacing an older entry. Because programs show *locality* (loops execute the same code, arrays are traversed sequentially, the stack is used near its top), hit ratios above 99% are common. On a context switch the TLB entries of the old process must be flushed, or tagged with an address-space identifier (ASID) so that they cannot be used by the new process.

Effective access time

Let α be the **hit ratio** (the fraction of accesses that find their page number in the TLB), t_m the time of one memory access and t_{TLB} the time of one TLB lookup.

Convention used in this course

We count the TLB lookup on every access, and a miss costs one extra memory access:

$$\text{EAT} = \alpha\,(t_{\text{TLB}} + t_m) + (1-\alpha)\,(t_{\text{TLB}} + 2t_m) = t_{\text{TLB}} + (2-\alpha)\,t_m.$$

Effective access time

Take $t_{\text{TLB}} = 20$ ns and $t_m = 100$ ns.

$$\alpha = 0.80: \quad \text{EAT} = 0.80 \times 120 + 0.20 \times 220 = 96 + 44 = 140 \text{ ns} \qquad (40\% \text{ slower than } t_m),$$
$$\alpha = 0.98: \quad \text{EAT} = 0.98 \times 120 + 0.02 \times 220 = 117.6 + 4.4 = 122 \text{ ns} \qquad (22\% \text{ slower}).$$

Without any TLB, every access costs $2t_m = 200$ ns. Check with the short form: $20 + (2 - 0.98) \times 100 = 122$ ns.

The modern simplification. On current CPUs the TLB lookup takes about one clock cycle and is overlapped with other work, so the 10th edition of Silberschatz ignores it ($t_{\text{TLB}} \approx 0$):

$$\text{EAT} = \alpha\, t_m + (1-\alpha)\, 2t_m.$$

With $t_m = 100$ ns this gives 120 ns for $\alpha = 0.80$ and 101 ns for $\alpha = 0.99$ (Silberschatz uses $t_m = 10$ ns: 12 ns and 10.1 ns). In an exam, always state which convention you use. All numbers in this section are checked by `code/verify.py`.

5.6 Protection bits and the valid--invalid bit

A page-table entry contains more than a frame number. Protection bits stored in each entry are checked by the MMU on every access, in parallel with the translation:

- A **valid–invalid bit**. *Valid* means the page belongs to the process's logical address space; *invalid* means it does not, and any access traps to the OS.
- **Read / write / execute** permissions. A write to a read-only page, or an attempt to execute instructions from a page marked no-execute (the NX bit of x86-64), also traps.

Page	Frame	Valid	Permissions	Contents
0	5	v	`r-x`	code
1	9	v	`r-x`	code
2	1	v	`rw-`	data
3	7	v	`rw-`	heap
4	12	v	`rw-`	stack
5–15	—	i	—	not part of the process

With this table, logical address `0x5000` (page 5) traps because the page is invalid, and a write into page 0 traps because code pages are read-only. The same letters `r`, `w`, `x` appear in `/proc/self/maps` (Section 7). Next week the valid bit will get a second use: a page may belong to the process but currently not be in RAM (a *page fault*, Week 7).

5.7 Shared pages

Paging makes it easy to share code. If code is **reentrant** (it never modifies itself), one copy in physical memory can be mapped into the page tables of many processes. A typical example is the C library `libc`: every process on a Linux machine uses it, but its code pages are in RAM only once. Only each process's private data pages are separate.

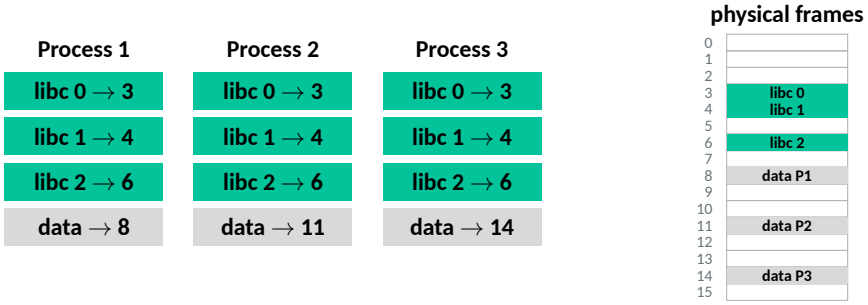


Figure 9: Three processes share the three code pages of the C library; only their data pages are private.

With 40 processes and a 2 MB library, sharing saves 39×2 MB $= 78$ MB. Shared pages are marked read-only so that no process can modify the common copy. The same mechanism lets `fork()` start by sharing *all* pages between parent and child, copying a page only when one of them writes to it (*copy-on-write*, Week 7).

6 Segmentation

6.1 The user's view of memory

Programmers do not think of their program as a sequence of 4 KB pages. They think of a main program, a set of functions, a stack, a symbol table, global data, and so on. Each of these is a logical unit of a different size, and within each unit locations are identified by their distance from its start ("the fifth instruction of `sqrt()`", "the top of the stack"). **Segmentation** is a memory-management scheme that supports exactly this view: the logical address space is a collection of **segments**, and a logical address is a pair

$$\langle \text{segment number } s,\ \text{offset } d \rangle.$$

The compiler creates the segments (for example: code, global variables, heap, one stack per thread, the standard C library), and the loader assigns each one a place in physical memory.

6.2 The segment table

The mapping is done by a **segment table**. Each entry has a **segment base** (the starting physical address of the segment) and a **segment limit** (its length). The offset d must lie between 0 and the limit; otherwise the hardware traps:

$$\text{if } 0 \le d < \text{limit}[s]: \quad a_{\text{phys}} = \text{base}[s] + d; \qquad \text{otherwise: trap (addressing error).}$$

In other words: segmentation is the base–limit idea of Section 2 with one pair of registers *per segment*.

Segment-table translation

s	Segment	Base	Limit	Occupies
0	main program	2000	1200	2000–3199
1	`sqrt()` library	6500	300	6500–6799
2	stack	4800	600	4800–5399
3	symbol table	3400	900	3400–4299
4	heap / data	5600	700	5600–6299

- $\langle 2, 53\rangle$: $53 < 600$, so $a_{\text{phys}} = 4800 + 53 = 4853$.
- $\langle 3, 852\rangle$: $852 < 900$, so $a_{\text{phys}} = 3400 + 852 = 4252$.
- $\langle 1, 299\rangle$: $299 < 300$, so $a_{\text{phys}} = 6500 + 299 = 6799$, the last byte of segment 1.
- $\langle 0, 1222\rangle$: $1222 \geq 1200$: **trap**. The program tried to access beyond the end of its main-program segment.

Because segments are units with a meaning, protection and sharing can be set per meaningful unit: code read-only and shareable, stack and data read-write and private. The price is that segments have variable sizes and must each be allocated contiguously: the problems of Section 4 (placement, external fragmentation, compaction) come back.

6.3 Paging versus segmentation

	Paging	Segmentation
Unit size	fixed (e.g. 4 KB), chosen by the hardware	variable, follows the program's structure
Visible to the programmer?	no	yes (main, stack, data, ...)
Logical address	one number, split into $p \mid d$	a pair $\langle s, d\rangle$
External fragmentation	none	yes
Internal fragmentation	yes, on average $\frac{1}{2}$ page per process	none
Protection and sharing	per page	per segment (meaningful unit)
Table entry	frame number and bits	base, limit and bits

Segments made of pages

Several historical systems combined the two schemes: the address space was divided into segments, and each segment was divided into pages. Multics (1960s, the ancestor of UNIX) worked this way, and so did the Intel 80386 (1985), whose 32-bit protected mode first translated a segmented address into a "linear" address and then paged it. Both mechanisms are still present in x86 processors for compatibility.

6.4 What x86-64 Linux actually does

In 64-bit mode, Intel and AMD processors essentially switch segmentation off: the code, data and stack segments are forced to base 0 and cover the whole address space (a *flat* memory model). Only the FS and GS segment registers keep a non-zero base, which Linux uses to find thread-local data. All real translation and protection is done by **paging**:

- four-level page tables for 48-bit virtual addresses ($9+9+9+9+12$ bits), or five levels (57 bits) on recent processors;
- page sizes of 4 KB, 2 MB and 1 GB;
- one set of page tables per process; the register CR3 holds the physical address of the top-level table, and a context switch (Week 3) reloads it.

ARM64 processors (phones, Apple M-series) have no segmentation at all. The *idea* of segments lives on in software: Linux keeps, for each process, a list of memory regions (program code, data, heap, stack, each shared library), and that is exactly what `/proc/<pid>/maps` prints. The message *Segmentation fault* survives as a name, although on Linux it now reports an access outside these regions or a violation of page permissions.

7 Hands-on: Real Addresses on Linux

The programs are in the `code/` folder next to these notes and compile with `gcc -Wall -O2` (or simply `make`). All output below was captured on our Linux test machine (x86-64). Because Linux randomises the position of the regions on every run (**ASLR**, address-space layout randomisation, a security measure), your addresses will be different, but the pattern will be the same.

7.1 Example 1: where do my variables live?

The helper `show_maps()` (see the full file `code/mem_layout.c`) prints the lines of `/proc/self/maps` that belong to the program file, the heap and the stack. The rest of the program:

```
int initialised_global = 42;   /* data segment             */
int uninitialised_global;      /* bss segment (zero-filled)  */
```

```
int main(void)
{
    int local = 7;                          /* stack                  */
    int *dynamic = malloc(sizeof(int));     /* heap                   */
    if (dynamic == NULL) return 1;
    *dynamic = 99;

    printf("text  (code)   main()      : %p\n", (void *)main);
    printf("data  (global) initialised : %p\n", (void *)&initialised_global);
    printf("bss   (global) zero-filled : %p\n", (void *)&uninitialised_global);
    printf("heap  (malloc)             : %p\n", (void *)dynamic);
    printf("stack (local variable)     : %p\n", (void *)&local);
    printf("page size                  : %ld bytes\n", sysconf(_SC_PAGESIZE));
    printf("\n--- my regions in /proc/self/maps ---\n");
    show_maps();
```

```
52
53      free(dynamic);
54      return 0;
55  }
```

```
$ ./mem_layout
text   (code)   main()      : 0x56412f3601e0
data   (global) initialised : 0x56412f363010
bss    (global) zero-filled : 0x56412f363018
heap   (malloc)             : 0x564160c6c2a0
stack  (local variable)     : 0x7ffe905b48c4
page size                   : 4096 bytes

--- my regions in /proc/self/maps ---
56412f35f000-56412f360000 r--p mem_layout
56412f360000-56412f361000 r-xp mem_layout
56412f361000-56412f362000 r--p mem_layout
56412f362000-56412f363000 r--p mem_layout
56412f363000-56412f364000 rw-p mem_layout
564160c6c000-564160c8d000 rw-p [heap]
7ffe90595000-7ffe905bb000 rw-p [stack]
```

Match each address with a region: `main` (`0x…3601e0`) lies in the `r-xp` region of the program file (readable and executable, not writable: code); the two globals lie in the `rw-p` region; the `malloc` block lies in `[heap]`; the local variable lies in `[stack]`, near the top of the user address space. Every region starts and ends at a multiple of `0x1000` $= 4096$: regions consist of whole pages. The heap region has `0x564160c8d000` $-$ `0x564160c6c000` $=$ `0x21000` $= 135\,168$ bytes $= 33$ pages; the stack region `0x26000` $= 155\,648$ bytes $= 38$ pages.

7.2 The page size and the memory map from the shell

```
$ getconf PAGESIZE
4096
$ grep Hugepagesize /proc/meminfo
Hugepagesize:       2048 kB
$ head -n 6 /proc/self/maps
559948881000-559948883000 r--p 00000000 fe:00 270606   /usr/bin/head
559948883000-559948889000 r-xp 00002000 fe:00 270606   /usr/bin/head
559948889000-55994888b000 r--p 00008000 fe:00 270606   /usr/bin/head
55994888b000-55994888c000 r--p 00009000 fe:00 270606   /usr/bin/head
55994888c000-55994888d000 rw-p 0000a000 fe:00 270606   /usr/bin/head
559970df7000-559970e18000 rw-p 00000000 00:00 0        [heap]
```

`/proc/self` always refers to the process that reads it, here head itself. The columns are: virtual start–end address, permissions (`r`, `w`, `x`, and `p` for private or `s` for shared), offset in the file, device, inode and file name. For example, the code region of head spans `0x559948889000` $-$ `0x559948883000` $=$ `0x6000` $= 24\,576$ bytes $= 6$ pages. (Spacing in the last column was shortened.)

7.3 Example 2: same address, different values

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int x = 100;                    /* one global variable */

int main(void)
{
    pid_t pid = fork();         /* now there are two processes */

    if (pid == 0) {             /* ---- child ---- */
        x = 200;                /* change OUR copy only        */
        printf("child  (PID %d): &x = %p   x = %d\n",
               getpid(), (void *)&x, x);
        return 0;
    }
    wait(NULL);                 /* ---- parent: wait for the child */
    printf("parent (PID %d): &x = %p   x = %d\n",
           getpid(), (void *)&x, x);
    return 0;
}
```

```
$ ./fork_vaddr
child  (PID 1760): &x = 0x55bc4a29c010   x = 200
parent (PID 1759): &x = 0x55bc4a29c010   x = 100
```

After `fork()` the child is a copy of the parent, so both have the same layout and &x is the same number in both. Yet the child changed x to 200 and the parent still sees 100. If &x were a physical address this would be impossible: one byte of RAM cannot hold two different values. The address is *logical*; each process has its own page table, which maps the page containing x to a different physical frame. (More precisely: right after `fork()` both page tables point to the same frame, marked read-only; when the child writes, the kernel copies the page and updates the child's entry. This copy-on-write technique is part of Week 7.) Run the program several times: because of ASLR the address changes from run to run, but parent and child always agree.

7.4 Example 3: page number and offset with bit operations

```c
#define N_BITS    12                    /* page size 2^12 = 4096 bytes */
#define PAGE_SIZE (1UL << N_BITS)
#define OFF_MASK  (PAGE_SIZE - 1)       /* 0xFFF: low 12 bits set      */

/* page table of a 16-page (64 KB) address space; -1 = invalid page */
static const int page_table[16] = { 5, 9, 1, 7, 12, -1, -1, -1,
                                    -1, -1, -1, -1, -1, -1, -1, -1 };

int main(int argc, char *argv[])
{
    printf("page size = %lu bytes, offset mask = 0x%lX\n\n",
```

```
           PAGE_SIZE, OFF_MASK);
    for (int i = 1; i < argc; i++) {
        unsigned long a = strtoul(argv[i], NULL, 0); /* accepts 0x.. */
        unsigned long p = a >> N_BITS;               /* high bits    */
        unsigned long d = a & OFF_MASK;              /* low 12 bits  */
        printf("0x%04lX (%5lu): p = %2lu, d = 0x%03lX (%4lu)", a, a, p, d, d);
        if (p >= 16 || page_table[p] < 0) {
            printf(" -> invalid: trap to OS\n");
            continue;
        }
        unsigned long f = (unsigned long)page_table[p];
        unsigned long phys = (f << N_BITS) | d;
        printf(" -> frame %2lu -> 0x%04lX (%5lu)\n", f, phys, phys);
    }
    return 0;
}
```

```
$ ./page_split 0x3A7F 0x1234 0x5000 0x0000
page size = 4096 bytes, offset mask = 0xFFF

0x3A7F (14975): p =  3, d = 0xA7F (2687) -> frame  7 -> 0x7A7F (31359)
0x1234 ( 4660): p =  1, d = 0x234 ( 564) -> frame  9 -> 0x9234 (37428)
0x5000 (20480): p =  5, d = 0x000 (   0) -> invalid: trap to OS
0x0000 (    0): p =  0, d = 0x000 (   0) -> frame  5 -> 0x5000 (20480)
```

The program does in software what the MMU does in hardware: shift right by 12 bits to obtain p, AND with `0xFFF` to obtain d, look up the frame and combine `(f << 12) | d`. The results agree with the worked example.

8 Summary

Week 6 in eight sentences

1. The OS must allocate RAM to many processes, protect them from each other and allow relocation; protection must be done by hardware.
2. Address binding can happen at compile, load or execution time; modern systems bind at execution time, so logical (virtual) and physical addresses differ and the MMU translates between them.
3. The simplest MMU checks $0 \leq a <$ limit and adds a base (relocation) register; a failed check traps.
4. Swapping whole processes takes seconds (100 MB at 50 MB/s: 2 s each way), so modern systems swap pages instead.
5. Contiguous allocation with first-, best- or worst-fit suffers from external fragmentation (50-percent rule: up to one third of memory lost); compaction helps but is expensive.
6. Paging divides memory into equal pages and frames; an address splits into p (high bits) and d (low bits); there is no external fragmentation, only internal fragmentation of half a page per process

on average.

7. Page tables are large (4 MB per process for 32-bit addresses), hence multi-level tables; a TLB caches translations, and EAT $= t_{\text{TLB}} + (2 - \alpha)\, t_m$ in our convention.
8. Segmentation translates $\langle s, d \rangle$ with a base and limit per segment; x86-64 Linux uses flat segments and does all the work with paging.

9 Glossary

Address binding
Mapping symbolic names to addresses; done at compile, load or execution time.

Backing store
Fast secondary storage that holds swapped-out memory.

Base (relocation) register
Holds the physical start address of a process; added to every logical address.

Best-fit / first-fit / worst-fit
Allocate the smallest / first / largest hole that is large enough.

Compaction
Moving allocated blocks together so that the free memory forms one block.

Effective access time (EAT)
Average time of one memory access, taking TLB hits and misses into account.

External fragmentation
Free memory split into holes that are each too small to be used.

Frame
A fixed-size block of physical memory.

Hole
A block of free memory between allocated blocks.

Internal fragmentation
Allocated but unused memory inside a block (e.g. the end of a process's last page).

Limit register
Holds the size of a process's logical address space; addresses $\geq$ limit trap.

Logical (virtual) address
An address generated by the CPU and seen by the program.

MMU
Memory-management unit: hardware that translates logical into physical addresses and checks protection.

Page
A fixed-size block of logical memory, the same size as a frame.

Page table
Per-process table mapping page numbers to frame numbers, with valid and protection bits.

Physical address
The address seen by the memory hardware.

Segment
A variable-size logical unit of a program (code, stack, data, ...).

Segment table
Table of (base, limit) pairs, one per segment.

Swapping
Moving a process (or, today, a page) between main memory and the backing store.

TLB
Translation look-aside buffer: small, fast associative cache of page-table entries.

Valid–invalid bit
Page-table bit that marks whether a page belongs to the process's address space.

10 Try It Yourself

Explore memory on your own machine

Linux (lab PCs, WSL, or a virtual machine):

```
$ getconf PAGESIZE            # page size in bytes
$ cat /proc/self/maps         # memory map of cat itself
$ make && ./mem_layout && ./mem_layout   # run twice: ASLR
$ free -h                     # RAM and swap in use
$ cat /proc/swaps             # the swap areas
$ vmstat 1 5                  # columns si/so = pages swapped in/out per second
```

Windows: Task Manager → *Performance* → *Memory* shows "In use", "Committed" and "Paged pool"; `systeminfo` prints the size of the page file (Windows' swap space).

1. What is the page size on your machine? (On an Apple-silicon Mac, `getconf PAGESIZE` prints 16384.)
2. How many pages large is the `[stack]` region of `cat`?
3. Do the addresses printed by `mem_layout` change between runs? Why does this make attacks harder?
4. Is your machine using swap right now? How much?

11 Review Questions

Conceptual

1. Name the three times at which address binding can take place. Which one requires special hardware, and why do modern systems use it?
2. What is the difference between a logical and a physical address? Who sees which?

3. Explain how base and limit registers protect processes from each other. Why must loading these registers be a privileged instruction?
4. Why do modern operating systems not swap whole processes?
5. Explain first-fit, best-fit and worst-fit. Which is usually fastest? Which tends to leave tiny holes?
6. Define external and internal fragmentation and give one example of each. Which one does paging eliminate?
7. State the 50-percent rule. What fraction of memory may be unusable?
8. Why must the page size be a power of two?
9. What problem does the TLB solve? Why are TLB hit ratios typically so high?
10. What is the valid–invalid bit used for? Give an access that would trap.
11. Why can code pages be shared between processes, but data pages normally not?
12. Compare paging and segmentation in three respects. What does x86-64 Linux use?
13. In `fork_vaddr`, parent and child print the same address but different values. Explain.

Numerical

14. Base $=$ `0x2000`, limit $=$ `0x0800`. Translate the logical addresses `0x07FF`, `0x0800` and `0x0123`. Give the legal physical range.
15. A process of 250 MB is swapped out and another of 150 MB swapped in; the disk transfers 100 MB/s. How long does the swap take (ignore latency)?
16. Holes of 100, 500, 200, 300, 600 KB (in this order). Requests: 212, 417, 112, 426 KB. Apply first-fit, best-fit and worst-fit. Which requests cannot be satisfied?
17. Pages are 4 KB, addresses 16 bits, page table $0 \to 5$, $1 \to 9$, $2 \to 1$, $3 \to 7$, $4 \to 12$. Translate `0x2ABC` and `0x4010`. Split `0x5C21` into p and d.
18. A process needs $72\,766$ bytes; pages are 2048 bytes. How many frames does it get, and how much internal fragmentation results?
19. Logical addresses have 32 bits, pages are 8 KB and page-table entries 4 bytes. How many entries does a flat page table have, and how large is it?
20. $t_{\text{TLB}} = 20$ ns, $t_m = 100$ ns, $\alpha = 0.90$. Compute the EAT with our convention and with the simplification $t_{\text{TLB}} = 0$. Which hit ratio is needed for an EAT of 125 ns (our convention)?
21. Using the segment table of Section 6.2, translate $\langle 4, 699\rangle$, $\langle 4, 700\rangle$ and $\langle 3, 0\rangle$.

Answers to the numerical questions

14. `0x07FF` $<$ `0x0800` $\Rightarrow$ `0x27FF`; `0x0800` is not $<$ limit $\Rightarrow$ trap; `0x0123` $\Rightarrow$ `0x2123`. Legal physical range `0x2000-0x27FF`.
15. $250/100 + 150/100 = 2.5 + 1.5 = 4$ s.

16. First-fit: $212 \to 500$ (leaves 288), $417 \to 600$ (leaves 183), $112 \to 288$ (leaves 176), 426 must wait. Best-fit: $212 \to 300$, $417 \to 500$, $112 \to 200$, $426 \to 600$: all fit. Worst-fit: $212 \to 600$ (leaves 388), $417 \to 500$, $112 \to 388$, 426 must wait.

17. `0x2ABC`: $p = 2$, $d =$ `0xABC`, frame $1 \Rightarrow$ `0x1ABC` $= 6844$. `0x4010`: $p = 4$, $d =$ `0x010`, frame $12 \Rightarrow$ `0xC010` $= 49168$. `0x5C21`: $p = 5$, $d =$ `0xC21` $= 3105$.

18. $72\,766 = 35 \times 2048 + 1086$, so 36 frames; internal fragmentation $2048 - 1086 = 962$ bytes.

19. $2^{32}/2^{13} = 2^{19} = 524\,288$ entries; $2^{19} \times 4$ B $= 2^{21}$ B $= 2$ MB.

20. Our convention: $20 + (2 - 0.9) \times 100 = 130$ ns ($= 0.9 \times 120 + 0.1 \times 220 = 108 + 22$). Simplified: $0.9 \times 100 + 0.1 \times 200 = 110$ ns. For 125 ns: $20 + (2 - \alpha)\,100 = 125 \Rightarrow \alpha = 0.95$.

21. $\langle 4, 699\rangle$: $699 < 700 \Rightarrow 5600 + 699 = 6299$. $\langle 4, 700\rangle$: $700 \geq 700 \Rightarrow$ trap. $\langle 3, 0\rangle$: 3400.

12 Looking Ahead: Week 7

- Virtual memory: a process does not need to be completely in RAM; demand paging and page faults.
- Copy-on-write after `fork()`.
- Page-replacement algorithms: FIFO, optimal (OPT) and LRU; thrashing.
- File systems: the file concept, access methods and directory structures.

Week 7 is the last lecture before the midterm exam in Week 8, which covers Weeks 1–7.

Further reading (optional)

- A. Silberschatz, P. B. Galvin and G. Gagne, *Operating System Concepts*, 10th ed., Wiley, Chapter 9 (Main Memory).
- R. H. and A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*, free online chapters:
  - `https://pages.cs.wisc.edu/~remzi/OSTEP/vm-intro.pdf` (Ch. 13, address spaces)
  - `https://pages.cs.wisc.edu/~remzi/OSTEP/vm-mechanism.pdf` (Ch. 15, base and bounds)
  - `https://pages.cs.wisc.edu/~remzi/OSTEP/vm-segmentation.pdf` (Ch. 16, segmentation)
  - `https://pages.cs.wisc.edu/~remzi/OSTEP/vm-freespace.pdf` (Ch. 17, free-space management)
  - `https://pages.cs.wisc.edu/~remzi/OSTEP/vm-paging.pdf` (Ch. 18, paging)
  - `https://pages.cs.wisc.edu/~remzi/OSTEP/vm-tlbs.pdf` (Ch. 19, TLBs)
  - `https://pages.cs.wisc.edu/~remzi/OSTEP/vm-smalltables.pdf` (Ch. 20, smaller page tables)